

INF-1400: Object-oriented programming

Mandatory assignment 2

Jens Christian Valen Leynse

March 20th, 2017

1 Introduction

The assignment was to create a simple simulator clone of the classic flock simulator boids. The code is written in python 3 using an object-oriented design including classes methods and the requirements for the assignment.

1.1 Requirements

1. The flock of boids should be able to move in a lifelike manner without hardcoding the movements.
2. The boids should follow set rules in their movements.
3. Obstacles that the boids should avoid are to be included.
4. A predator (hoik) should hunt the boids and try to eat them.
5. A class diagram of the object-oriented design is to be included in the report.
6. The code design is to include exceptions.
7. The code is to include an object-oriented technique from class.

2 Technical Background

The code utilizes the object-oriented technique inheritance. As the definitions of the superclass apply to more than one class, the information of the definitions is inherited to the classes categorized as subclasses. Three python libraries are included in the code; pygame, random and time.

3 Design & Implementation

The code is separated into eight classes; four of them are the objects added into the simulator (more specifically the boids, hoiks, obstacles and the bait). One class consists of the rules for the movements of the moving objects on the screen. Another consists of all the functions that alter the positioning and direction of the moving objects. The class that controls an objects existence is a class of lists, and lastly the game class is where a loop that holds the simulator active is.

All of the object classes take in a position. If there is movement involved a speed direction is added as well as a radius that determines some priorities for the object at later stages. The obstacle class also has its own draw definition since the definition for the other objects to be drawn is set to different figure.

The information for all of the objects are collected in lists in a class. Here the mouse coordinates get sent to when the object is appended to their respectable lists. An exception is raised if the number of objects reaches over a fixed amount. An exception to the list definition structures is the bait definition that checks to see if there is already an object in the list before adding a new one. An activation variable is sent to the bait definition determining whether the list is to have the new object or not. Lastly the definitions take all of the lists and sends them to the another class for position and direction changes, by iterating through each list. The boids and hoiks are thereby sent to a movement definition, and the boids, hoiks and obstacles are sent to the different draw definitions (together with a color and the screen surface).

The flyer class is where most of the changes is made to the objects. There are six different definitions for the class and in order they are;
Draw: Uses the position and direction of the object to draw a triangle.

Move boid: collects all of the lists, sends the boids to the family definition, then uses the different lists to make different priority vectors. All of the vectors are then added to the directional speed vector of the objects. The vector is normalized as to reset the vector length and then multiplied to a variable determining the speed.

Move hoik: uses much of the same definition calls as the movement definition for the boids but doesn't use all of the rules and has different priorities.

Family: Checks the distance between the boids in the boids list sent to the definition. If the distance is smaller than the radius determined in the objects class, the object is added to a group and follows a different set of rules than the more general group of boids.

Remove: Checks the position of the hoiks compared to the boids. Seeing as how the hoiks are hunting the boids, the boids are removed from the list of boids if the hoiks hits (comes within a very little radius around the boid).

Mirror border: Is just a wall making definition that makes it so that the objects can't leave the screen. The position of the object is

switched to the other side of the screen so that the movements doesn't need to be reversed.

The most important class when it comes to the objects movement is the rulebook. Here the boids and hoiks gets their information sent to give them more life like movement as well as different priorities to follow.

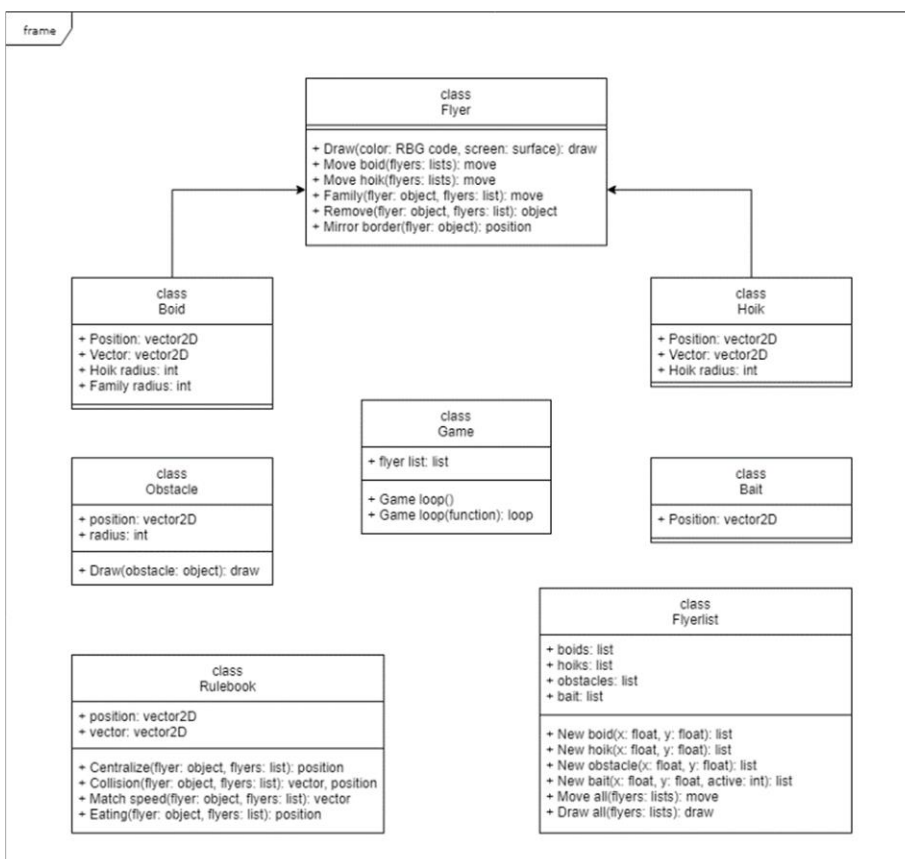
Three rules were already determined in the assignment requirements, but one extra was added for the hunting mechanism since a better alteration was found. These rules are:

Centralize: Makes it so that the objects in the list gets a vector to move towards the center position of all of the objects. If there is only one object not much is changed, adding one more makes the objects gather in groups.

Collision: Makes the objects avoid each other and all of the objects from a sent in list. This definition takes the distance between the objects and changes the vector based on the proximity of others. A new vector is returned if the distance is too small.

Match speed: This definition is only directed towards the groups and makes it so that the objects try to match the speed direction of each other. Again this definition is only relevant to a group, so single objects are not affected.

Eating: This is the definition added by myself as an alteration of the centralize definition. The definition checks to see which object is the closest and gives a vector to another object towards that location. The definition is used in both the vector the hoiks use to track and hunt a boid, as well as the vector the boids get if a bait object is introduced.

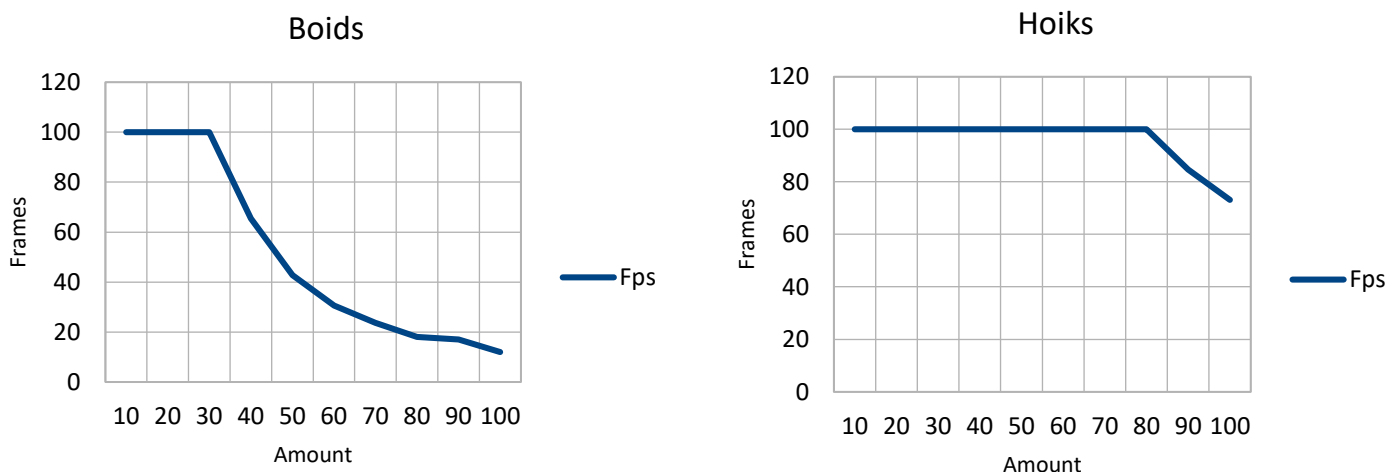


As shown in the UML diagram, the hoik class and the boid class inherits the methods from the flyer class.

All of the variables utilized in the code is imported from a configuration file.

5 Discussion

A method to rotate the triangles was tried implemented, and as a result; the objects movement got smoother. Unfortunately, the speed directional vector was changed with this method making the objects increase in speed uncontrollably. Any attempt at slowing down the speed could not retain the movements, be it normalizing the vector or adding a definition to force a speed limit upon the vector to not pass a certain point wielded no positive results, so the method ended up not being implemented. A reoccurring problem in the code was how the number of objects affected each other and created lag. Several tests was performed to maximize the efficiency and as shown underneath; the boids slowed down the framerate of the simulation by a significant amount given an increased amount of them. Since the hoiks utilized fewer rules the framerate impact was less.



An exception was raised to stop the code from appending more objects to the list. And the reason the fps never goes over a hundred is because of a fps cap, set in the game class of the code.

6 Conclusion

All of the requirements for the assignment is met. The boids move in a semi life like manner with their rules applied, as well as the obstacles and hoiks. The code raises exceptions given the right circumstances, and a object-oriented technique is implemented.

References

The pygame library with its many functions included

<https://pygame.org/docs/>

A descriptive page on how a boid works

<http://www.red3d.com/cwr/boids/>

A pseudocode of the boids movements

<http://www.kfish.org/boids/pseudocode.html>

Another description of the rules for the boids (differently explained)

<http://code.activestate.com/recipes/502216-boids-demonstration/>